Modélisation du projet

Gestion des vols et des réservations

Agence de voyage de Cotonou

Ce document présente la modélisation orientée objet de l'application console de gestion des vols, des réservations, des paiements et des annulations : classes, attributs et méthodes, associations, multiplicités et diagramme de classes UML.

1. Identification des classes

On repère les noms importants de l'énoncé :

Nom dans l'énoncé	Classe retenue	Justification
passager	**Passager**	Entité avec des données propres (identité, contact)
vol	**Vol**	Entité avec une capacité, un prix et un horaire
réservation	**Reservation**	Relie un passager à un vol et possède un statut
paiement, remboursement	**Paiement**	Possède un montant, un mode et un statut de remboursement
agence	**Agence**	Classe de gestion : stocke les collections et applique les règles

Le « billet » n'a pas de classe propre : il correspond à la réservation validée. Les statuts sont modélisés par deux énumérations : `StatutReservation` (EN_ATTENTE, VALIDEE, ANNULEE) et `StatutRemboursement` (AUCUN, REMBOURSE_PARTIEL).

2. Attributs et méthodes de chaque classe

Passager

Attributs (privés)	Méthodes (publiques)
- id : int	+ Passager(id, nom, prenom, tel, email)
- nom : string	+ getId() : int
- prenom : string	+ nomComplet() : string
- telephone : string	+ afficher() : void
- email : string	

Vol

Attributs (privés)	Méthodes (publiques)
- numero : string	+ Vol(numero, depart, destination, date,
- depart : string	heure, capacite, prix)
- destination : string	+ placesDisponibles() : int
- date : string	+ estComplet() : bool
- heure : string	+ occuperPlace() : void
- capacite : int	+ libererPlace() : void
- placesOccupees : int	+ afficher() : void
- prix : double	

Reservation

Attributs (privés)	Méthodes (publiques)
- id : int - statut : StatutReservation - passager : Passager* - vol : Vol* - paiement : Paiement*	+ Reservation(id, passager, vol) + payer(paiement) : void + annuler() : void + estAnnulee() : bool + estPayee() : bool + afficherResume() : void + afficherDetails() : void

Paiement

Attributs (privés)	Méthodes (publiques)
- id : int - montant : double - mode : string - statutRemboursement : StatutRemboursement - montantRembourse : double	+ Paiement(id, montant, mode) + rembourser() : void

Agence

Attributs (privés)	Méthodes (publiques)
- nom : string - passagers : vector<Passager*> - vols : vector<Vol*> - reservations : vector<Reservation*>	+ enregistrerPassager(...) : int + ajouterVol(...) : void + afficherVolsDisponibles() : void + rechercherVol(critere) : void + reserver(idPassager, numeroVol) : int + enregistrerPaiement(idReservation, mode) : void + annulerReservation(idReservation) : void + consulterReservations() : void + afficherInfosReservation(id) : void

3. Associations entre les classes

Association	Nom du lien	Type	Commentaire
Agence → Passager	« enregistre »	Agrégation	L'agence conserve ses passagers
Agence → Vol	« propose »	Agrégation	L'agence conserve ses vols
Agence → Reservation	« gère »	Composition	Une réservation n'existe pas sans l'agence
Reservation → Passager	« effectuée par »	Association	Une réservation connaît son passager
Reservation → Vol	« concerne »	Association	Une réservation connaît son vol
Reservation → Paiement	« est réglée par »	Composition	Le paiement n'existe pas sans sa réservation

Deux dépendances relient aussi `Reservation` à `StatutReservation` et `Paiement` à `StatutRemboursement`.

4. Multiplicités

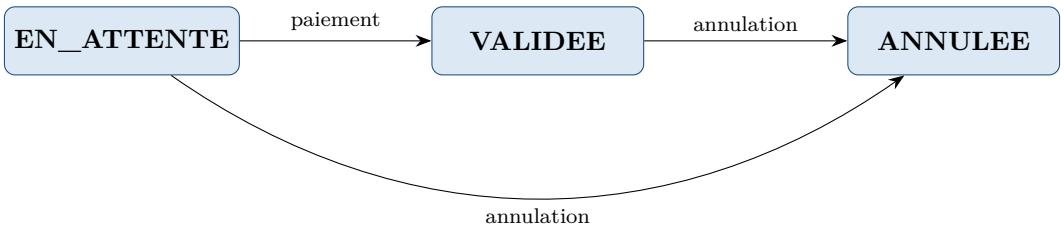
Association	Gauche	Droite	Justification
Agence – Passager	1	0..*	Une agence a de 0 à plusieurs passagers
Agence – Vol	1	0..*	Une agence propose de 0 à plusieurs vols
Agence – Reservation	1	0..*	Une agence gère de 0 à plusieurs réservations
Passager – Reservation	1	0..*	Un passager peut avoir plusieurs réservations, sur des vols différents (règle 3)
Vol – Reservation	1	0..*	Un vol reçoit au plus autant de réservations que sa capacité (règle 2)
Reservation – Paiement	1	0..1	Aucun paiement tant que la réservation est en attente, un seul ensuite

Vue depuis la réservation, la multiplicité est toujours **1** pour le passager et pour le vol : c'est la règle 4.

5. Règles de gestion et classes responsables

Règle	Appliquée dans
1. Passager enregistré avant de réserver	`Agence::reserver()`
2. Capacité du vol respectée	`Vol::occuperPlace()`
3. Pas de double réservation du même vol	`Agence::reserver()`
4. Un passager et un vol par réservation	Constructeur de `Reservation`
5. Réservation annulée non payable	`Reservation::payer()`
6. Validation après paiement	`Reservation::payer()`
7. Remboursement à l'annulation (80 %)	`Reservation::annuler()`, `Paiement::rembourser()`

Cycle de vie d'une réservation



6. Diagramme de classes UML

